

Inside the Transformer: The Math, Step by Step

A fully worked numeric example through all 8 modules

Companion deep-dive to “What Is an LLM, and What Are Transformer Models?” — June 2026

Thirumalai Parthasarathy • VP Delivery, Onward Technologies, Chicago

The setup

We push one sentence through a **single Transformer block** and watch the numbers. To keep every matrix printable we use small “teaching” dimensions; real models use the same equations at vastly larger scale.

Sentence (9 tokens): Weak quarterly results led the board to replace the
Model dim $d_{\text{model}} = 8$ **Heads** $H = 2$ **Per-head** $d_k = d_{\text{model}}/H = 4$ **FFN width** $d_{\text{ff}} = 16$

Every token becomes a row vector of length 8. Stacking the 9 tokens gives a 9×8 matrix X . All values are rounded to 2 decimals; tiny rounding drift is expected if you recompute by hand. Weight matrices ($W_Q, W_K, W_V, \dots$) are fixed numbers a real model *learns* during training — here they are illustrative. **Notation.** A token is a row. “ $\odot$ ” is element-wise multiply. “ $M^\top$ ” is transpose. Matrix multiply $A_{(a \times b)} B_{(b \times c)} = C_{(a \times c)}$.

Module 1 — Embedding (+ position)

A lookup table turns each token id into a learned vector. The result is our input matrix $X \in \mathbb{R}^{9 \times 8}$ (rows = the 9 words, top to bottom):

$$X = \begin{bmatrix} -0.85 & 0.56 & -0.12 & 0.45 & 0.96 & 0.08 & 0.00 & -0.86 \\ -0.46 & -0.00 & 0.36 & 0.61 & -0.24 & -0.87 & -0.42 & 0.82 \\ -0.57 & -0.10 & 0.86 & -0.95 & 0.20 & 0.90 & -0.54 & 0.10 \\ 0.82 & -0.73 & 0.05 & 0.50 & 0.34 & -0.06 & -0.59 & -0.02 \\ -0.26 & -0.05 & -0.27 & 0.68 & 0.54 & -0.37 & 0.15 & -0.45 \\ -0.09 & -0.29 & 0.31 & -0.26 & -0.08 & 0.44 & -0.17 & 0.81 \\ -0.64 & 0.48 & -0.16 & -0.15 & 0.27 & 0.05 & -0.17 & -1.00 \\ -0.82 & 0.42 & 0.05 & 0.39 & 0.91 & 0.37 & -0.89 & -0.38 \\ 0.19 & -0.53 & 0.93 & 0.89 & 0.70 & -0.06 & 0.68 & -0.74 \end{bmatrix}$$

Position is injected later, *inside* attention, via RoPE (Module 3). The embedding itself carries only *meaning*.

Module 2 — Layer Normalization

The classic LayerNorm rescales each row so the network stays numerically stable. For a row vector $x = (x_1, \dots, x_d)$:

$$\mu = \frac{1}{d} \sum_{j=1}^d x_j, \quad \sigma^2 = \frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2, \quad \hat{x}_j = \frac{x_j - \mu}{\sqrt{\sigma^2 + \epsilon}} \gamma_j + \beta_j.$$

In words: *subtract the mean from each value, divide by the square root of the variance*, then scale (γ) and shift (β). **What modern (2025–2026) models actually use: RMSNorm.** It drops the mean-subtraction and the β shift, normalizing only by the root-mean-square magnitude — cheaper, and empirically just as stable:

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2 + \epsilon}, \quad \hat{x}_j = \frac{x_j}{\text{RMS}(x)} g_j.$$

Worked on the first token, “Weak” = $(-0.85, 0.56, -0.12, 0.45, 0.96, 0.08, 0.00, -0.86)$, with gain $g = 1$:

$$\frac{1}{8} \sum x_j^2 = 0.365, \quad \text{RMS} = \sqrt{0.365 + \epsilon} = 0.604,$$

$$\hat{x} = \frac{1}{0.604}(-0.85, \dots, -0.86) = (-1.41, 0.93, -0.20, 0.74, 1.59, 0.13, 0.00, -1.42).$$

Doing this for all 9 rows gives the normalized matrix X_n that feeds attention:

$$X_n = \text{RMSNorm}(X) = \begin{bmatrix} -1.41 & 0.93 & -0.20 & 0.74 & 1.59 & 0.13 & 0.00 & -1.42 \\ -0.84 & -0.00 & 0.66 & 1.12 & -0.44 & -1.60 & -0.77 & 1.50 \\ -0.91 & -0.16 & 1.38 & -1.52 & 0.32 & 1.44 & -0.86 & 0.16 \\ 1.67 & -1.49 & 0.10 & 1.02 & 0.69 & -0.12 & -1.20 & -0.04 \\ -0.66 & -0.13 & -0.68 & 1.71 & 1.36 & -0.93 & 0.38 & -1.13 \\ -0.24 & -0.77 & 0.82 & -0.69 & -0.21 & 1.17 & -0.45 & 2.14 \\ -1.35 & 1.01 & -0.34 & -0.32 & 0.57 & 0.11 & -0.36 & -2.11 \\ -1.36 & 0.70 & 0.08 & 0.65 & 1.51 & 0.61 & -1.48 & -0.63 \\ 0.29 & -0.80 & 1.41 & 1.35 & 1.06 & -0.09 & 1.03 & -1.12 \end{bmatrix}$$

Module 3 — Multi-Head Attention

This is the engine. It runs in four moves: **(a)** project to Query/Key/Value, **(b)** add position with RoPE, **(c)** score & mix per head, **(d)** recombine.

(a) Project to Q, K, V via W_Q, W_K, W_V

Each learned matrix is 8×8 . We multiply the normalized input by each:

$$Q = X_n W_Q, \quad K = X_n W_K, \quad V = X_n W_V.$$

The three weight matrices (a real model’s “learned” parameters):

$$W_Q = \begin{bmatrix} -0.23 & -0.04 & 0.29 & -0.02 & -0.44 & -0.19 & -0.21 & -0.24 \\ -0.40 & -0.10 & -0.06 & 0.33 & 0.36 & 0.03 & -0.05 & 0.33 \\ 0.46 & 0.21 & 0.36 & 0.53 & -0.55 & 0.45 & -0.27 & -0.03 \\ 0.36 & 0.26 & -0.42 & 0.19 & -0.52 & -0.17 & 0.38 & -0.09 \\ 0.12 & 0.27 & 0.39 & 0.31 & -0.59 & -0.10 & -0.04 & -0.53 \\ 0.05 & 0.13 & 0.39 & 0.53 & -0.45 & -0.32 & 0.19 & -0.44 \\ -0.33 & 0.09 & -0.40 & 0.34 & 0.43 & -0.56 & 0.04 & 0.36 \\ 0.57 & -0.27 & -0.40 & 0.45 & 0.49 & -0.36 & -0.07 & 0.26 \end{bmatrix} \quad W_K = \begin{bmatrix} 0.41 & -0.40 & 0.20 & 0.37 & 0.06 & -0.40 & -0.56 & -0.26 \\ 0.37 & -0.55 & -0.59 & -0.17 & -0.52 & -0.42 & -0.57 & 0.03 \\ 0.24 & -0.09 & -0.44 & -0.20 & 0.11 & 0.53 & 0.59 & -0.31 \\ -0.59 & 0.40 & 0.51 & -0.05 & 0.33 & 0.44 & 0.13 & 0.45 \\ -0.57 & -0.27 & -0.27 & -0.46 & 0.49 & -0.56 & 0.21 & -0.51 \\ -0.17 & -0.10 & -0.38 & 0.03 & 0.04 & -0.22 & 0.28 & -0.41 \\ -0.37 & -0.17 & -0.15 & -0.35 & 0.50 & 0.39 & -0.47 & -0.16 \\ -0.32 & -0.06 & -0.27 & 0.00 & 0.51 & -0.14 & 0.18 & 0.11 \end{bmatrix}$$

$$W_V = \begin{bmatrix} 0.30 & -0.53 & 0.29 & 0.54 & 0.12 & -0.25 & 0.21 & 0.25 \\ 0.19 & -0.42 & 0.57 & 0.55 & -0.09 & 0.11 & -0.55 & 0.59 \\ 0.38 & 0.16 & 0.31 & -0.37 & -0.23 & -0.30 & 0.12 & -0.49 \\ 0.47 & -0.05 & -0.07 & -0.47 & 0.22 & 0.38 & 0.16 & -0.31 \\ 0.34 & -0.43 & 0.39 & 0.10 & -0.25 & 0.02 & 0.15 & -0.29 \\ 0.42 & -0.09 & 0.47 & 0.40 & -0.48 & 0.18 & -0.23 & 0.30 \\ 0.05 & -0.05 & 0.47 & -0.53 & 0.07 & -0.21 & -0.56 & 0.30 \\ 0.07 & 0.47 & 0.12 & -0.20 & 0.58 & -0.46 & -0.54 & 0.28 \end{bmatrix}$$

Carrying out $Q = X_n W_Q$ etc. (each 9×8):

$$Q = \begin{bmatrix} -0.49 & 0.94 & 0.39 & 0.29 & -1.01 & 0.39 & 0.65 & -0.68 \\ 1.88 & -0.34 & -1.56 & 0.01 & 0.81 & 0.71 & 0.00 & 1.13 \\ 0.85 & 0.10 & 1.85 & 1.05 & -0.75 & 0.98 & -0.54 & -0.81 \\ 1.08 & 0.44 & 0.90 & -0.55 & -2.75 & 0.17 & -0.01 & -1.74 \\ -0.15 & 0.93 & -0.68 & -0.52 & -1.05 & -0.12 & 0.84 & -0.49 \\ 1.89 & -0.44 & 0.26 & 1.42 & 0.19 & -0.36 & -0.33 & -0.17 \\ -1.38 & 0.50 & 0.81 & -0.72 & -0.26 & 1.06 & 0.33 & -0.33 \\ 0.64 & 0.69 & 0.99 & 0.43 & -1.64 & 0.91 & 0.52 & -1.27 \\ 0.53 & 1.39 & 0.49 & 0.86 & -2.58 & 0.08 & 0.17 & -0.94 \end{bmatrix} \quad K = \begin{bmatrix} -1.19 & 0.01 & -0.46 & -1.40 & -0.29 & -0.33 & 0.35 & -0.23 \\ -0.52 & 1.04 & 0.55 & -0.07 & 0.49 & 1.27 & 1.10 & 1.69 \\ 0.64 & -0.37 & -2.02 & -0.31 & -0.46 & -0.36 & 2.12 & -1.48 \\ -0.36 & 0.58 & 1.74 & 0.90 & 0.94 & -0.36 & 0.77 & -0.17 \\ -1.89 & 0.81 & 1.35 & -0.96 & 0.76 & 0.46 & -0.09 & 0.65 \\ -0.38 & 0.06 & -1.20 & 0.20 & 1.06 & -0.06 & 1.85 & -0.59 \\ 0.39 & -0.09 & -0.45 & -0.72 & -1.72 & -0.39 & -0.12 & -0.17 \\ -0.88 & 0.23 & -0.64 & -0.83 & -0.52 & -0.89 & 1.56 & -0.21 \\ -1.25 & 0.35 & 0.49 & -0.96 & 1.49 & 1.55 & 0.81 & -0.72 \end{bmatrix}$$

$$V = \begin{bmatrix} 0.52 & -1.08 & 0.52 & -0.03 & -1.33 & 1.50 & 0.26 & -0.75 \\ -0.23 & 1.57 & -1.22 & -1.80 & 1.69 & -0.39 & 0.01 & -1.04 \\ 0.19 & 0.70 & 0.60 & 0.66 & -1.49 & -0.41 & -0.07 & -0.40 \\ 0.86 & -0.54 & -0.76 & 0.23 & 0.31 & 0.04 & 2.17 & -1.43 \\ 0.33 & -0.84 & -0.46 & -1.19 & -0.06 & 1.30 & 0.94 & -1.31 \\ 0.32 & 1.63 & 0.31 & -0.27 & 0.40 & -1.22 & -0.84 & 0.17 \\ -0.42 & -0.98 & -0.05 & 0.82 & -1.69 & 1.51 & 0.47 & -0.31 \\ 0.71 & -0.52 & 0.09 & 0.62 & -1.24 & 1.38 & 0.70 & -1.04 \\ 1.40 & -0.69 & 0.69 & -1.69 & -0.72 & 0.23 & 1.09 & -1.85 \end{bmatrix}$$

Intuition: the **Query** is “what am I looking for,” the **Key** is “what do I offer,” the **Value** is “what I’ll hand over if matched.”

(b) Inject position with RoPE

Rotary Positional Embedding rotates each *pair* of dimensions of Q and K (not V) by an angle that grows with the token’s position p . For pair i (dims $2i, 2i+1$):

$$\theta_{p,i} = p \cdot 10000^{-2i/d}, \quad \begin{bmatrix} x'_{2i} \\ x'_{2i+1} \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} x_{2i} \\ x_{2i+1} \end{bmatrix}.$$

Position 0 is unrotated; later positions turn more. After applying RoPE, Q and K become:

$$Q^{\text{rope}} = \begin{bmatrix} -0.49 & 0.94 & 0.39 & 0.29 & -1.01 & 0.39 & 0.65 & -0.68 \\ 1.30 & 1.40 & -1.55 & -0.15 & 0.80 & 0.72 & -0.00 & 1.13 \\ -0.44 & 0.73 & 1.60 & 1.40 & -0.77 & 0.96 & -0.54 & -0.81 \\ -1.13 & -0.28 & 1.02 & -0.26 & -2.75 & 0.09 & -0.00 & -1.74 \\ 0.80 & -0.49 & -0.42 & -0.74 & -1.04 & -0.16 & 0.84 & -0.49 \\ 0.11 & -1.94 & -0.45 & 1.37 & 0.21 & -0.35 & -0.33 & -0.17 \\ -1.19 & 0.87 & 1.08 & -0.14 & -0.32 & 1.04 & 0.33 & -0.33 \\ 0.03 & 0.94 & 0.48 & 0.97 & -1.70 & 0.79 & 0.53 & -1.27 \\ -1.45 & 0.32 & -0.28 & 0.95 & -2.58 & -0.13 & 0.18 & -0.94 \end{bmatrix} \quad K^{\text{rope}} = \begin{bmatrix} -1.19 & 0.01 & -0.46 & -1.40 & -0.29 & -0.33 & 0.35 & -0.23 \\ -1.16 & 0.12 & 0.55 & -0.01 & 0.48 & 1.27 & 1.10 & 1.69 \\ 0.07 & 0.74 & -1.92 & -0.71 & -0.45 & -0.37 & 2.12 & -1.48 \\ 0.27 & -0.62 & 1.40 & 1.37 & 0.95 & -0.33 & 0.77 & -0.17 \\ 1.85 & 0.90 & 1.62 & -0.36 & 0.74 & 0.49 & -0.09 & 0.65 \\ -0.05 & 0.38 & -1.15 & -0.40 & 1.06 & -0.01 & 1.85 & -0.58 \\ 0.35 & -0.20 & 0.04 & -0.85 & -1.69 & -0.49 & -0.12 & -0.17 \\ -0.81 & -0.40 & 0.05 & -1.05 & -0.46 & -0.92 & 1.56 & -0.20 \\ -0.16 & -1.29 & 1.03 & -0.32 & 1.36 & 1.66 & 0.82 & -0.71 \end{bmatrix}$$

(Row 1 is identical to Q above because $p = 0$; later rows differ.)

(c) Scores, causal mask, softmax — per head

Split the 8 columns into $H = 2$ heads: head 1 = columns 1–4, head 2 = columns 5–8. For each head, raw attention scores compare every Query against every Key:

$$S = \frac{Q_h^{\text{rope}} (K_h^{\text{rope}})^\top}{\sqrt{d_k}}, \quad \sqrt{d_k} = \sqrt{4} = 2.$$

For **head 1**, S is 9×9 (row = querying token, column = token being looked at):

$$S^{(1)} = \begin{bmatrix} 0.00 & 0.45 & -0.15 & 0.11 & 0.23 & -0.09 & -0.30 & -0.13 & -0.41 \\ -0.30 & -1.10 & 2.10 & -1.45 & 0.60 & 1.15 & 0.12 & -0.77 & -1.78 \\ -1.08 & 0.73 & -1.78 & 1.79 & 0.97 & -1.05 & -0.71 & -0.66 & 0.16 \\ 0.62 & 0.92 & -1.03 & 0.47 & -0.30 & -0.56 & -0.04 & 0.68 & 0.84 \\ 0.14 & -0.61 & 0.51 & -0.54 & 0.31 & 0.28 & 0.50 & 0.15 & 0.15 \\ -0.93 & -0.31 & -0.77 & 1.24 & -1.38 & -0.39 & -0.38 & -0.39 & 0.79 \\ 0.56 & 1.04 & -0.71 & 0.23 & 0.19 & -0.40 & -0.21 & 0.41 & 0.11 \\ -0.80 & 0.17 & -0.46 & 0.71 & 0.66 & -0.29 & -0.49 & -0.70 & -0.52 \\ 0.26 & 0.78 & -0.00 & 0.16 & -1.60 & 0.07 & -0.70 & 0.02 & -0.39 \end{bmatrix}$$

Causal mask: a token may only look at itself and earlier tokens, so we set every entry *above* the diagonal to $-\infty$ before softmax (those become 0 weight). Then softmax each row (subtract row-max, exponentiate, divide by the sum):

$$\alpha_{ij} = \frac{\exp(S_{ij})}{\sum_{k \leq i} \exp(S_{ik})}.$$

Head 1 attention weights $A^{(1)}$ (each row sums to 1; upper triangle is 0 from the mask):

$$A^{(1)} = \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.69 & 0.31 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.13 & 0.80 & 0.07 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.29 & 0.40 & 0.06 & 0.25 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.22 & 0.10 & 0.31 & 0.11 & 0.26 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.07 & 0.12 & 0.08 & 0.58 & 0.04 & 0.11 & 0.00 & 0.00 & 0.00 \\ 0.19 & 0.31 & 0.05 & 0.14 & 0.13 & 0.07 & 0.09 & 0.00 & 0.00 \\ 0.06 & 0.15 & 0.08 & 0.25 & 0.24 & 0.09 & 0.08 & 0.06 & 0.00 \\ 0.14 & 0.24 & 0.11 & 0.13 & 0.02 & 0.12 & 0.05 & 0.11 & 0.07 \end{bmatrix}$$

Head 2 attention weights $A^{(2)}$ (a *different* learned pattern over the same words):

$$A^{(2)} = \begin{bmatrix} 1.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.12 & 0.88 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.37 & 0.22 & 0.40 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.20 & 0.01 & 0.75 & 0.03 & 0.00 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.18 & 0.09 & 0.56 & 0.11 & 0.07 & 0.00 & 0.00 & 0.00 & 0.00 \\ 0.19 & 0.12 & 0.15 & 0.20 & 0.18 & 0.16 & 0.00 & 0.00 & 0.00 \\ 0.12 & 0.19 & 0.19 & 0.10 & 0.12 & 0.15 & 0.12 & 0.00 & 0.00 \\ 0.09 & 0.03 & 0.38 & 0.04 & 0.03 & 0.06 & 0.25 & 0.12 & 0.00 \\ 0.09 & 0.01 & 0.22 & 0.02 & 0.01 & 0.02 & 0.49 & 0.12 & 0.01 \end{bmatrix}$$

Each head's output is its weights times its Values, $O_h = A^{(h)}V_h$ (each 9×4):

$$O^{(1)} = \begin{bmatrix} 0.52 & -1.08 & 0.52 & -0.03 \\ 0.29 & -0.25 & -0.02 & -0.58 \\ -0.10 & 1.17 & -0.87 & -1.41 \\ 0.29 & 0.21 & -0.49 & -0.63 \\ 0.33 & -0.13 & -0.03 & -0.27 \\ 0.57 & 0.01 & -0.49 & -0.12 \\ 0.19 & 0.17 & -0.40 & -0.61 \\ 0.35 & -0.07 & -0.37 & -0.36 \\ 0.36 & 0.24 & -0.16 & -0.40 \end{bmatrix} \quad O^{(2)} = \begin{bmatrix} -1.33 & 1.50 & 0.26 & -0.75 \\ 1.32 & -0.16 & 0.04 & -1.00 \\ -0.72 & 0.31 & 0.07 & -0.67 \\ -1.35 & -0.01 & 0.08 & -0.52 \\ -0.88 & 0.09 & 0.31 & -0.69 \\ -0.17 & 0.22 & 0.50 & -0.81 \\ -0.24 & 0.18 & 0.28 & -0.68 \\ -1.16 & 0.47 & 0.25 & -0.53 \\ -1.41 & 0.94 & 0.37 & -0.51 \end{bmatrix}$$

(d) Concatenate heads, project with W_O

Glue the head outputs back side by side into a 9×8 matrix, then mix them with the learned output projection $W_O \in \mathbb{R}^{8 \times 8}$:

$$O = [O^{(1)} \mid O^{(2)}], \quad \text{Attn} = O W_O.$$

$$O = \begin{bmatrix} 0.52 & -1.08 & 0.52 & -0.03 & -1.33 & 1.50 & 0.26 & -0.75 \\ 0.29 & -0.25 & -0.02 & -0.58 & 1.32 & -0.16 & 0.04 & -1.00 \\ -0.10 & 1.17 & -0.87 & -1.41 & -0.72 & 0.31 & 0.07 & -0.67 \\ 0.29 & 0.21 & -0.49 & -0.63 & -1.35 & -0.01 & 0.08 & -0.52 \\ 0.33 & -0.13 & -0.03 & -0.27 & -0.88 & 0.09 & 0.31 & -0.69 \\ 0.57 & 0.01 & -0.49 & -0.12 & -0.17 & 0.22 & 0.50 & -0.81 \\ 0.19 & 0.17 & -0.40 & -0.61 & -0.24 & 0.18 & 0.28 & -0.68 \\ 0.35 & -0.07 & -0.37 & -0.36 & -1.16 & 0.47 & 0.25 & -0.53 \\ 0.36 & 0.24 & -0.16 & -0.40 & -1.41 & 0.94 & 0.37 & -0.51 \end{bmatrix}$$

$$\text{Attn} = O W_O = \begin{bmatrix} -0.43 & -0.01 & 1.09 & -0.03 & 0.89 & -0.24 & 1.42 & -0.43 \\ 0.75 & -0.70 & 0.10 & 0.29 & 1.50 & 0.20 & -0.56 & 1.14 \\ 0.79 & 1.04 & 0.76 & 0.26 & -0.05 & 1.54 & -0.74 & 1.06 \\ 0.60 & 0.65 & 0.91 & -0.27 & -0.37 & 0.64 & 0.43 & -0.39 \\ 0.43 & 0.38 & 0.76 & -0.50 & 0.07 & 0.21 & 0.67 & -0.21 \\ 0.54 & -0.19 & 0.61 & -0.26 & 0.25 & 0.30 & 0.14 & 0.21 \\ 0.63 & 0.21 & 0.57 & 0.01 & 0.30 & 0.56 & -0.10 & 0.52 \\ 0.32 & 0.35 & 0.86 & -0.14 & -0.11 & 0.40 & 0.54 & -0.27 \\ -0.06 & 0.76 & 1.00 & -0.28 & -0.04 & 0.61 & 0.54 & -0.06 \end{bmatrix}$$

Module 4 — Dropout

During *training only*, dropout zeroes a random fraction p of entries and scales the rest by $1/(1-p)$ so the average signal is unchanged:

$$\text{Dropout}(z)_j = \begin{cases} \frac{z_j}{1-p} & \text{with prob. } 1-p \\ 0 & \text{with prob. } p \end{cases}$$

At **inference** (what we are doing) dropout is the identity — it changes nothing — so Attn passes through unchanged. Its only job is to stop the model leaning on any single pathway during training.

Module 5 — Residual (skip) connection

Add the attention result back to the block's *original* input. The sub-layer only had to learn an *adjustment*:

$$Z = X + \text{Attn}$$

$$Z = \begin{bmatrix} -1.28 & 0.55 & 0.97 & 0.42 & 1.85 & -0.16 & 1.42 & -1.29 \\ 0.29 & -0.70 & 0.46 & 0.90 & 1.26 & -0.67 & -0.98 & 1.96 \\ 0.22 & 0.94 & 1.62 & -0.69 & 0.15 & 2.44 & -1.28 & 1.16 \\ 1.42 & -0.08 & 0.96 & 0.23 & -0.03 & 0.58 & -0.16 & -0.41 \\ 0.17 & 0.33 & 0.49 & 0.18 & 0.61 & -0.16 & 0.82 & -0.66 \\ 0.45 & -0.48 & 0.92 & -0.52 & 0.17 & 0.74 & -0.03 & 1.02 \\ -0.01 & 0.69 & 0.41 & -0.14 & 0.57 & 0.61 & -0.27 & -0.48 \\ -0.50 & 0.77 & 0.91 & 0.25 & 0.80 & 0.77 & -0.35 & -0.65 \\ 0.13 & 0.23 & 1.93 & 0.61 & 0.66 & 0.55 & 1.22 & -0.80 \end{bmatrix}$$

This shortcut is what lets 100+ layers train without the signal vanishing.

Module 6 — Layer Normalization (again)

The feed-forward sub-layer gets its own pre-norm, identical formula to Module 2:

$$Z_n = \text{RMSNorm}(Z)$$

$$Z_n = \begin{bmatrix} -1.13 & 0.49 & 0.86 & 0.37 & 1.64 & -0.14 & 1.26 & -1.14 \\ 0.28 & -0.68 & 0.45 & 0.88 & 1.23 & -0.65 & -0.95 & 1.91 \\ 0.17 & 0.74 & 1.27 & -0.54 & 0.12 & 1.91 & -1.00 & 0.91 \\ 2.14 & -0.12 & 1.45 & 0.35 & -0.05 & 0.87 & -0.24 & -0.62 \\ 0.35 & 0.67 & 1.00 & 0.37 & 1.25 & -0.33 & 1.68 & -1.35 \\ 0.72 & -0.76 & 1.46 & -0.83 & 0.27 & 1.18 & -0.05 & 1.62 \\ -0.02 & 1.51 & 0.90 & -0.31 & 1.25 & 1.34 & -0.59 & -1.05 \\ -0.75 & 1.16 & 1.37 & 0.38 & 1.21 & 1.16 & -0.53 & -0.98 \\ 0.14 & 0.25 & 2.06 & 0.65 & 0.70 & 0.59 & 1.30 & -0.85 \end{bmatrix}$$

Module 7 — Feed-Forward Network (SwiGLU)

Now each token is processed *independently* through a small two-layer network. The 2025–2026 standard is the gated **SwiGLU**, which uses *two* up-projections — a “gate” and a “value” — combined with the SiLU/Swish activation $\text{SiLU}(x) = x \sigma(x) = \frac{x}{1 + e^{-x}}$:

$$\text{FFN}(z) = \underbrace{\left(\text{SiLU}(zW_{\text{gate}}) \right)}_{\text{the gate}} \odot \underbrace{(zW_{\text{up}})}_{\text{the value}} W_{\text{down}}.$$

Shapes: $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{8 \times 16}$ expand to the wide hidden layer; $W_{\text{down}} \in \mathbb{R}^{16 \times 8}$ compresses back. The gate W_{gate} :

$$W_{\text{gate}} = \begin{bmatrix} 0.44 & -0.18 & 0.39 & 0.02 & -0.46 & 0.29 & -0.45 & 0.33 & -0.49 & 0.17 & -0.33 & -0.16 & 0.45 & -0.01 & 0.16 & 0.24 \\ -0.39 & 0.34 & 0.41 & -0.34 & 0.05 & -0.22 & 0.24 & -0.47 & 0.01 & 0.29 & 0.23 & -0.39 & 0.33 & -0.26 & 0.31 & -0.00 \\ -0.34 & 0.23 & 0.31 & 0.20 & 0.46 & -0.24 & -0.26 & -0.35 & 0.04 & -0.10 & -0.14 & -0.04 & -0.23 & -0.50 & -0.03 & -0.22 \\ -0.05 & 0.43 & -0.32 & 0.14 & 0.14 & -0.22 & -0.03 & 0.47 & -0.16 & 0.21 & 0.34 & -0.14 & 0.49 & 0.13 & 0.00 & 0.21 \\ -0.08 & 0.21 & -0.23 & 0.43 & 0.04 & -0.26 & -0.06 & 0.40 & 0.31 & -0.02 & -0.15 & -0.12 & 0.48 & -0.39 & 0.39 & 0.36 \\ -0.28 & -0.15 & 0.07 & 0.30 & 0.45 & 0.49 & -0.22 & -0.31 & -0.01 & 0.06 & 0.16 & -0.39 & 0.15 & 0.42 & 0.42 & 0.44 \\ 0.42 & 0.46 & -0.45 & 0.42 & -0.23 & -0.18 & -0.04 & 0.46 & 0.11 & -0.15 & -0.13 & 0.40 & 0.32 & -0.36 & -0.25 & -0.13 \\ -0.28 & 0.06 & -0.28 & 0.07 & 0.44 & 0.35 & -0.40 & -0.29 & -0.09 & 0.15 & -0.11 & -0.25 & -0.16 & -0.30 & 0.36 & 0.20 \end{bmatrix}$$

Applying the full SwiGLU to every row yields $F \in \mathbb{R}^{9 \times 8}$:

$$F = \text{FFN}(Z_n) = \begin{bmatrix} -0.23 & -0.47 & 0.05 & 0.18 & -0.09 & 0.66 & 0.25 & 0.43 \\ -0.37 & 0.23 & -0.77 & 0.48 & 0.39 & 0.60 & -0.16 & -0.32 \\ -1.38 & 0.45 & -1.23 & 0.68 & 0.41 & 1.06 & 0.99 & -0.84 \\ -0.14 & -0.41 & -0.04 & 0.13 & 0.45 & -0.02 & 0.08 & 0.22 \\ -0.17 & -0.32 & 0.31 & 0.16 & -0.47 & 0.11 & -0.05 & 0.08 \\ -1.07 & 0.51 & -1.04 & 0.73 & 0.24 & 1.05 & 0.46 & -0.42 \\ -0.64 & -1.00 & 0.27 & 0.28 & 0.60 & 0.43 & 0.33 & -0.60 \\ -0.99 & -0.61 & -0.09 & 0.33 & 0.33 & 0.77 & 0.13 & -0.70 \\ -0.35 & -0.49 & 0.49 & 0.29 & -0.25 & -0.04 & -0.41 & 0.09 \end{bmatrix}$$

2026 reality: in frontier models this single FFN is replaced by a **Mixture-of-Experts** — dozens of expert FFNs plus a router that sends each token to only a few. Same equation, run sparsely: huge capacity, a fraction of the compute.

Module 8 — Residual → output

A second residual add finishes the block:

$$Y = Z + F$$

$$Y = \begin{bmatrix} -1.51 & 0.08 & 1.02 & 0.60 & 1.76 & 0.50 & 1.67 & -0.86 \\ -0.08 & -0.47 & -0.31 & 1.38 & 1.65 & -0.07 & -1.14 & 1.64 \\ -1.16 & 1.39 & 0.39 & -0.01 & 0.56 & 3.50 & -0.29 & 0.32 \\ 1.28 & -0.49 & 0.92 & 0.36 & 0.42 & 0.56 & -0.08 & -0.19 \\ 0.00 & 0.01 & 0.80 & 0.34 & 0.14 & -0.05 & 0.77 & -0.58 \\ -0.62 & 0.03 & -0.12 & 0.21 & 0.41 & 1.79 & 0.43 & 0.60 \\ -0.65 & -0.31 & 0.68 & 0.14 & 1.17 & 1.04 & 0.06 & -1.08 \\ -1.49 & 0.16 & 0.82 & 0.58 & 1.13 & 1.54 & -0.22 & -1.35 \\ -0.22 & -0.26 & 2.42 & 0.90 & 0.41 & 0.51 & 0.81 & -0.71 \end{bmatrix}$$

Y is the block's output; it becomes the *input* to the next block. After the **final** block, one last RMSNorm and the language-model head turn the last token's vector into next-word probabilities.

Final norm and the LM head

Normalize, then project the *last* row (the position predicting the next word) onto the vocabulary with W_{vocab} . Using a 6-word demo vocabulary [CEO, chief, leadership, CFO, board, weather]:

$$Y_n = \text{RMSNorm}(Y), \quad \text{logits} = W_{\text{vocab}} (Y_n)_{\text{last}}^{\top}.$$

The last token's normalized vector is $(Y_n)_{\text{last}} = (-0.22, -0.25, 2.37, 0.88, 0.40, 0.50, 0.79, -0.69)$. The logits and their softmax:

$$\text{logits} = [4.53 \quad 2.74 \quad 0.13 \quad 1.12 \quad -0.92 \quad -0.07], \quad \text{softmax} \Rightarrow [0.82 \quad 0.14 \quad 0.01 \quad 0.03 \quad 0.00 \quad 0.01]$$

$$P(\text{CEO}) = 0.82 \quad P(\text{chief}) = 0.14 \quad P(\text{CFO}) = 0.03 \quad \text{others} \approx 0$$

The model completes “Weak quarterly results led the board to replace the **CEO**.”

The chosen token is appended to the input and the entire pipeline runs again for the next word. That loop — predict, append, repeat — is **autoregression**.

Counting the parameters

Per block, the learned numbers are:

$$\underbrace{4d^2}_{W_Q, W_K, W_V, W_O} + \underbrace{3dd_{\text{ff}}}_{\text{SwiGLU}} + \underbrace{2d}_{\text{two norms}} = 4(64) + 3(8)(16) + 16 = 256 + 384 + 16 = 656.$$

A real model with $d=4096$, $d_{\text{ff}} \approx 14000$ and ~ 100 such blocks reaches tens of billions of parameters — but every one of them is doing exactly the arithmetic above.

All matrices computed with NumPy and rounded to 2 decimals; weights are illustrative, not from a trained model. The output projection was aligned so the demo lands on “CEO,” which is what a *trained* LM head learns to do. Companion to the main article and LinkedIn carousel.